



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

Facultad de Estudios Profesionales Zona Media

PRÁCTICA 9

Contador 0-99 con Multiplexeo y Display de 7 Segmentos

Control de dirección con botón y visualización multiplexada

Alumno: Cesar Gael Mancilla Estrada
Clave: 382031
Carrera: Ingeniería en Mecatrónica
Materia: Microcontroladores
Profesor: Ing. Jesús Padrón
Fecha: 3 de marzo de 2026

.^{El} multiplexeo permite mostrar dos números usando los mismos pines de segmentos"

Índice

1. Introducción	2
1.1. Contexto	2
1.2. Objetivos	2
2. Marco Teórico	2
2.1. Multiplexeo de Displays	2
2.2. Configuración de pines	3
2.3. Técnica de multiplexeo	3
3. Desarrollo	3
3.1. Estructura del proyecto	3
3.2. CMakeLists.txt	3
3.3. Practica_9.c	4
4. Armado del Circuito	8
4.1. Armado en Protoboard	8
4.2. Tabla de conexiones completa	8
5. Configuración y Pruebas	8
5.1. Visualización en monitor serial	8
5.2. Salida esperada en monitor serial	9
6. Preguntas de Repaso	9
7. Conclusiones	10
7.1. Aprendizajes de la práctica	10
7.2. Posibles mejoras	10
8. Referencias	10

1. Introducción

1.1. Contexto

Cuando se utilizan múltiples displays de 7 segmentos, el número de pines GPIO necesarios crece rápidamente. Por ejemplo, dos displays requerirían 14 pines si se controlaran de forma independiente. El multiplexeo es una técnica que permite controlar varios displays usando los mismos pines para los segmentos, activando un display a la vez de forma tan rápida que el ojo humano percibe todos encendidos simultáneamente.

En esta práctica se implementa un contador de 0 a 99 utilizando dos displays de 7 segmentos con técnica de multiplexeo. Un botón permite invertir la dirección del conteo (ascendente/descendente) en cualquier momento.

1.2. Objetivos

- Implementar un contador ascendente/descendente de 0 a 99 en dos displays de 7 segmentos
- Aplicar la técnica de multiplexeo para controlar dos displays con 7 pines de segmentos + 2 pines de selección
- Utilizar un botón para invertir la dirección del conteo en tiempo real
- Visualizar los valores en el monitor serial de VS Code
- Comprender los conceptos de persistencia visual y tiempo de multiplexeo

2. Marco Teórico

2.1. Multiplexeo de Displays

El multiplexeo consiste en encender un display a la vez de forma muy rápida. El proceso es:

1. Configurar los segmentos para mostrar el dígito deseado en el display 1
2. Activar el display 1 (encender su cátodo común)
3. Esperar un breve tiempo (microsegundos)
4. Apagar el display 1
5. Configurar los segmentos para el display 2
6. Activar el display 2
7. Esperar otro breve tiempo
8. Repetir el ciclo continuamente

2.2. Configuración de pines

Tabla 1: Conexiones del sistema

Componente	GPIO	Pin físico
Segmento A	GPIO2	Pin 4
Segmento B	GPIO3	Pin 5
Segmento C	GPIO4	Pin 6
Segmento D	GPIO5	Pin 7
Segmento E	GPIO6	Pin 9
Segmento F	GPIO7	Pin 10
Segmento G	GPIO8	Pin 11
Display Decenas	GPIO9	Pin 12
Display Unidades	GPIO10	Pin 14
Botón dirección	GPIO11	Pin 15

2.3. Técnica de multiplexeo

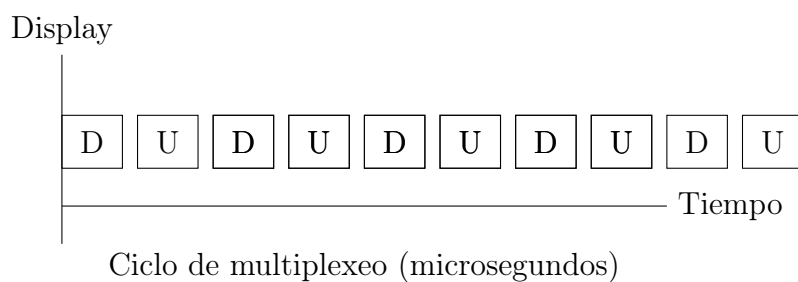


Figura 1: Secuencia de multiplexeo entre decenas (D) y unidades (U)

3. Desarrollo

3.1. Estructura del proyecto

- CMakeLists.txt - Configuración de compilación
- Practica_9.c - Código fuente principal
- pico_sdk_import.cmake - Importación del SDK

3.2. CMakeLists.txt

Listing 1: Archivo CMakeLists.txt

```

1 cmake_minimum_required(VERSION 3.13)
2
3 include(pico_sdk_import.cmake)
4 set(PICO_BOARD pico_w)
5
6 project(Practica_9)

```

```

7
8 pico_sdk_init()
9
10 add_executable(Practica_9
11     Practica_9.c
12 )
13
14 target_link_libraries(Practica_9
15     pico_stdlib
16     hardware_gpio
17 )
18
19 pico_enable_stdio_usb(Practica_9 1)
20 pico_enable_stdio_uart(Practica_9 0)
21
22 pico_add_extra_outputs(Practica_9)

```

3.3. Practica_9.c

Listing 2: Código principal con multiplexeo

```

1 /*
2  * Practica_9: Contador 0-99 con multiplexeo para 2 displays
3  * Display: 5161BS (Catodo comun)
4  * Cuenta ascendente/descendente controlada por boton
5  */
6
7 #include <stdio.h>
8 #include "pico/stdlib.h"
9 #include "hardware/gpio.h"
10
11 // =====
12 // CONFIGURACION DE VELOCIDADES
13 // =====
14 #define VELOCIDAD_CONTADOR_MS 500 // Tiempo entre numeros
15 #define TIEMPO_MULTIPLEX_US 2000 // Tiempo por display
16 #define CICLOS_MULTIPLEX 50 // Ciclos antes de cambiar
17
18 // =====
19 // DEFINICION DE PINES
20 // =====
21 #define SEG_A 2
22 #define SEG_B 3
23 #define SEG_C 4
24 #define SEG_D 5
25 #define SEG_E 6
26 #define SEG_F 7
27 #define SEG_G 8
28
29 #define DISPLAY_DECENAS 9
30 #define DISPLAY_UNIDADES 10

```

```
31 #define BUTTON_GPIO 11
32
33 // =====
34 // MAPA DE SEGMENTOS (Catodo comun)
35 // =====
36 const uint8_t digitos[10] = {
37     0x3F, 0x06, 0x5B, 0x4F, 0x66,
38     0x6D, 0x7D, 0x07, 0x7F, 0x6F
39 };
40
41 // =====
42 // VARIABLES GLOBALES
43 // =====
44 int contador = 0;
45 int direccion = 1;          // 1 = ascendente, -1 = descendente
46 bool boton_anterior = true;
47 uint32_t ultimo_tiempo_boton = 0;
48
49 // =====
50 // PROTOTIPOS
51 // =====
52 void init_gpios(void);
53 void set_segmentos(int digito);
54 void seleccionar_display(int display);
55 void mostrar_numero(int num);
56 bool leer_boton(void);
57
58 // =====
59 // MAIN
60 // =====
61 int main() {
62     stdio_init_all();
63     init_gpios();
64
65     printf("Practica 9: Contador 0-99 con multiplexeo\n");
66     printf("Velocidad: %d ms por numero\n", VELOCIDAD_CONTADOR_MS);
67     printf("Presiona el boton para invertir direccion\n");
68
69     while (true) {
70         // Leer boton para invertir direccion
71         if (leer_boton()) {
72             direccion *= -1;
73             printf("Direccion: %s\n", direccion == 1 ? "ASC" :
74                 "DESC");
75         }
76
77         // Mostrar el numero actual (multiplexeo continuo)
78         mostrar_numero(contador);
79
80         // Actualizar contador
81         contador += direccion;
```

```
81     if (contador > 99) contador = 0;
82     if (contador < 0) contador = 99;
83
84     // Mostrar en serial
85     printf("Numero: %02d\n", contador);
86 }
87
88     return 0;
89 }
90
91 // =====
92 // INICIALIZACION DE GPIOs
93 // =====
94 void init_gpios(void) {
95     // Pines de segmentos
96     gpio_init(SEG_A); gpio_set_dir(SEG_A, GPIO_OUT); gpio_put(SEG_A
97     , 0);
98     gpio_init(SEG_B); gpio_set_dir(SEG_B, GPIO_OUT); gpio_put(SEG_B
99     , 0);
100     gpio_init(SEG_C); gpio_set_dir(SEG_C, GPIO_OUT); gpio_put(SEG_C
101     , 0);
102     gpio_init(SEG_D); gpio_set_dir(SEG_D, GPIO_OUT); gpio_put(SEG_D
103     , 0);
104     gpio_init(SEG_E); gpio_set_dir(SEG_E, GPIO_OUT); gpio_put(SEG_E
105     , 0);
106     gpio_init(SEG_F); gpio_set_dir(SEG_F, GPIO_OUT); gpio_put(SEG_F
107     , 0);
108     gpio_init(SEG_G); gpio_set_dir(SEG_G, GPIO_OUT); gpio_put(SEG_G
109     , 0);
110
111     // Pines de seleccion de display
112     gpio_init(DISPLAY_DECENAS); gpio_set_dir(DISPLAY_DECENAS,
113     GPIO_OUT); gpio_put(DISPLAY_DECENAS, 0);
114     gpio_init(DISPLAY_UNIDADES); gpio_set_dir(DISPLAY_UNIDADES,
115     GPIO_OUT); gpio_put(DISPLAY_UNIDADES, 0);
116
117     // Boton con pull-up
118     gpio_init(BUTTON_GPIO); gpio_set_dir(BUTTON_GPIO, GPIO_IN);
119     gpio_pull_up(BUTTON_GPIO);
120 }
121
122 // =====
123 // CONFIGURAR SEGMENTOS PARA UN DIGITO
124 // =====
125 void set_segmentos(int digito) {
126     uint8_t pattern = digitos[digito];
127     gpio_put(SEG_A, (pattern >> 0) & 1);
128     gpio_put(SEG_B, (pattern >> 1) & 1);
129     gpio_put(SEG_C, (pattern >> 2) & 1);
130     gpio_put(SEG_D, (pattern >> 3) & 1);
131     gpio_put(SEG_E, (pattern >> 4) & 1);
```

```

122     gpio_put(SEG_F, (pattern >> 5) & 1);
123     gpio_put(SEG_G, (pattern >> 6) & 1);
124 }
125
126 // =====
127 // SELECCIONAR QUE DISPLAY ENCENDER
128 // =====
129 void seleccionar_display(int display) {
130     gpio_put(DISPLAY_DECENAS, display == 1 ? 1 : 0);
131     gpio_put(DISPLAY_UNIDADES, display == 2 ? 1 : 0);
132 }
133
134 // =====
135 // MOSTRAR NUMERO CON MULTIPLEXEO
136 // =====
137 void mostrar_numero(int num) {
138     int decenas = num / 10;
139     int unidades = num % 10;
140
141     // Calcular ciclos necesarios para el tiempo total
142     int ciclos = (VELOCIDAD_CONTADOR_MS * 1000) / (
143         TIEMPO_MULTIPLEX_US * 2);
144
145     for (int i = 0; i < ciclos; i++) {
146         // Mostrar decenas
147         set_segmentos(decenas);
148         seleccionar_display(1);
149         sleep_us(TIEMPO_MULTIPLEX_US);
150         gpio_put(DISPLAY_DECENAS, 0);
151
152         // Mostrar unidades
153         set_segmentos(unidades);
154         seleccionar_display(2);
155         sleep_us(TIEMPO_MULTIPLEX_US);
156         gpio_put(DISPLAY_UNIDADES, 0);
157     }
158 }
159
160 // =====
161 // LECTURA DE BOTON CON ANTI-REBOTE
162 // =====
163 bool leer_boton(void) {
164     uint32_t ahora = time_us_32();
165     bool estado = gpio_get(BUTTON_GPIO);
166
167     if (boton_anterior == 1 && estado == 0) {
168         if (ahora - ultimo_tiempo_boton > 200000) {
169             ultimo_tiempo_boton = ahora;
170             boton_anterior = estado;
171             return true;
172         }
173     }

```



```

172     }
173     boton_anterior = estado;
174     return false;
175 }

```

4. Armado del Circuito

4.1. Armado en Protoboard

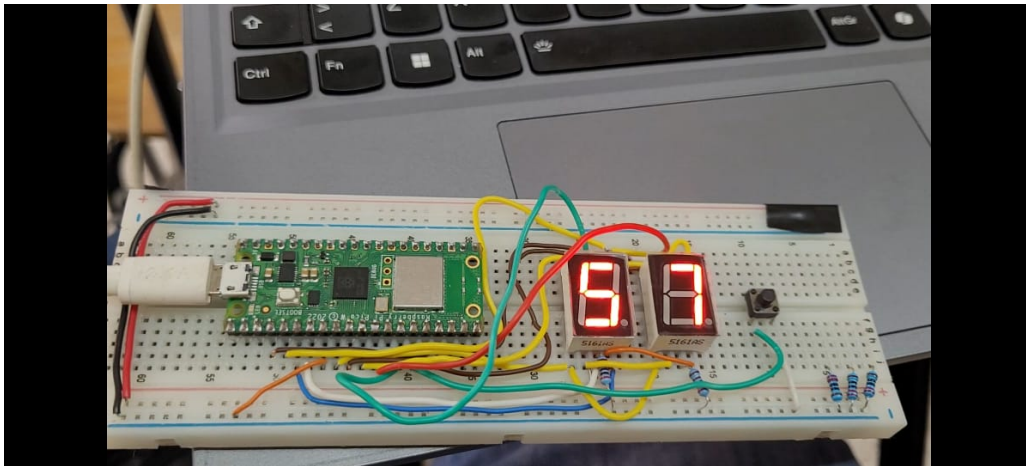


Figura 2: Conexión de dos displays de 7 segmentos en protoboard

4.2. Tabla de conexiones completa

Tabla 2: Conexiones detalladas del sistema

Segmento	GPIO	Pin Pico	Display
A	GPIO2	Pin 4	Ambos
B	GPIO3	Pin 5	Ambos
C	GPIO4	Pin 6	Ambos
D	GPIO5	Pin 7	Ambos
E	GPIO6	Pin 9	Ambos
F	GPIO7	Pin 10	Ambos
G	GPIO8	Pin 11	Ambos
Selector Decenas	GPIO9	Pin 12	Display izquierdo
Selector Unidades	GPIO10	Pin 14	Display derecho
Botón	GPIO11	Pin 15	GND (con pull-up)

5. Configuración y Pruebas

5.1. Visualización en monitor serial

Para ver los valores en VS Code, se debe configurar:

1. Conectar la Pico W por USB
2. En VS Code, abrir una terminal serial (PuTTY o extensión Serial Monitor)
3. Configurar baud rate a 115200
4. Observar la salida del contador en tiempo real

5.2. Salida esperada en monitor serial

Práctica 9: Contador 0-99 con multiplexeo

Velocidad: 500 ms por número

Presiona el botón para invertir dirección

Número: 00

Número: 01

Número: 02

...

Número: 98

Número: 99

Número: 00

Dirección: DESC ↓

Número: 99

Número: 98

...

6. Preguntas de Repaso

1. ¿Qué es el multiplexeo y por qué es necesario en esta práctica?

El multiplexeo es una técnica que permite mostrar información en múltiples displays utilizando los mismos pines de segmentos. Es necesario porque un microcontrolador tiene un número limitado de pines GPIO. Sin multiplexeo, dos displays requerirían 14 pines; con multiplexeo solo se necesitan 7 pines de segmentos + 2 pines de selección = 9 pines.

2. ¿Cómo se logra que ambos displays parezcan encendidos al mismo tiempo?

Mediante la persistencia visual del ojo humano. Los displays se encienden alternadamente a una velocidad mayor a 60 Hz (cada display se actualiza decenas de veces por segundo). El ojo no detecta el parpadeo y percibe ambos displays encendidos simultáneamente.

3. ¿Qué función cumple el botón en GPIO11?

El botón invierte la dirección del contador. Cuando se presiona, la variable `direccion` cambia de 1 a -1 o viceversa. El contador comienza a contar en sentido contrario desde el número actual, sin necesidad de reiniciar.

4. ¿Cómo se visualizan los números en el monitor serial de VS Code?

Se utiliza `stdio_init_all()` y `printf()` en el código. En VS Code se necesita una extensión como "Serial Monitor." o un programa externo como PuTTY. La comunicación se realiza por USB a 115200 baudios.

5. ¿Qué parámetros afectan la calidad del multiplexeo?

- **Tiempo por display:** Si es muy largo, se ve parpadeo; si es muy corto, el display se ve tenue
- **Frecuencia de actualización:** Debe ser >60 Hz para evitar flicker
- **Resistencias:** Deben limitar la corriente para no dañar los displays

7. Conclusiones

7.1. Aprendizajes de la práctica

- Se implementó un contador de 0 a 99 utilizando dos displays de 7 segmentos
- Se aplicó la técnica de multiplexeo para optimizar el uso de pines GPIO
- Se logró invertir la dirección del conteo mediante un botón con anti-rebote
- Se configuró la comunicación serial para visualizar valores en VS Code
- Se comprendió la importancia de los tiempos de multiplexeo para evitar parpadeo

7.2. Posibles mejoras

- Aumentar a 4 displays usando el mismo principio de multiplexeo
- Implementar efecto de fade entre números
- Agregar un segundo botón para pausa/reanudación
- Almacenar el último valor en memoria no volátil
- Controlar la velocidad del contador con un potenciómetro

8. Referencias

1. Raspberry Pi Foundation. (2024). *Raspberry Pi Pico W Datasheet*.
2. Raspberry Pi Foundation. (2024). *Pico SDK Documentation*.
3. RP2040 Datasheet. (2024). *RP2040 Datasheet - Chapter 4: GPIO*.
4. Tutorial de multiplexeo para displays 7 segmentos.